



Powered by
Arizona State University®

Universidad Internacional del Ecuador

Ingeniería en Inteligencia Artificial

Lógica de programación

Evaluación en Contacto con el Docente

Oscar Ivan Castro Urrea

1er Semestre

28 de Junio de 2025

**REINVENTEMOS
EL FUTURO**

1. Introducción

El presente documento describe el desarrollo del proyecto final de la asignatura Lógica de Programación, correspondiente al programa académico de la Universidad Internacional del Ecuador (UIDE). El proyecto consiste en la implementación del clásico videojuego Atari Pong utilizando el lenguaje de programación Python y la biblioteca Pygame.

Atari Pong fue seleccionado como proyecto porque permite aplicar de forma práctica los conceptos fundamentales de la programación: variables, estructuras de control condicionales y repetitivas, funciones, manejo de eventos e interacción con el usuario. Al mismo tiempo, su lógica es suficientemente clara para ser representada mediante diagramas de flujo y arquitectura de software.

A lo largo de las ocho semanas de desarrollo se siguió un cronograma estructurado que abarcó la planificación, el diseño, la implementación y las pruebas del sistema, utilizando GitHub como herramienta de control de versiones.

2. Descripción del Problema

Se requiere desarrollar un programa interactivo que recree el funcionamiento básico del videojuego Atari Pong. El sistema debe ser capaz de gestionar todos los elementos del juego de forma autónoma y responder a la entrada del usuario en tiempo real.

2.1 Definición del Alcance

El sistema a desarrollar debe cumplir con los siguientes requisitos funcionales:

- Mostrar una pantalla de juego con dos paletas y una pelota en movimiento continuo.
- Permitir que dos jugadores controlen sus respectivas paletas mediante el teclado.
- Detectar colisiones de la pelota con las paletas y los bordes superior e inferior.
- Asignar puntos automáticamente cuando la pelota supera la paleta del oponente.
- Mostrar el marcador actualizado en pantalla durante toda la partida.
- Determinar y mostrar al ganador cuando algún jugador alcanza 5 puntos.

2.2 Metodología de Resolución del Problema

Para abordar el problema se siguió la metodología clásica de resolución de problemas en programación:

1. Identificar el problema: recrear Atari Pong con control de paletas, pelota y marcador.



Powered by

Arizona State University®

2. Comprender el problema: analizar la mecánica del juego, sus reglas y la interacción requerida.
3. Identificar soluciones: evaluar alternativas de implementación (pygame, turtle, tkinter).
4. Seleccionar la mejor solución: pygame por su soporte nativo para gráficos en tiempo real.
5. Listar los pasos de la solución: diseñar los diagramas de flujo y arquitectura.
6. Evaluar y probar la solución: ejecutar pruebas funcionales y corregir errores detectados.

3. Relación con los Contenidos de la Asignatura

El proyecto integra de manera directa los temas abordados durante las ocho semanas de la asignatura. A continuación se describe cómo cada contenido se evidencia en el sistema desarrollado.

3.1 Variables y Tipos de Datos

El sistema utiliza variables de tipo entero para almacenar los marcadores de cada jugador (score1, score2), listas para representar la velocidad vectorial de la pelota (ball_speed = [5, 5]), y objetos Rect de pygame para definir la posición y dimensiones de la pelota y las paletas.

3.2 Estructuras Condicionales

Las estructuras condicionales (if / elif / else) son el núcleo de la lógica del juego. Se utilizan para detectar colisiones con bordes, verificar si la pelota salió del área de juego, comprobar la condición de victoria y procesar la entrada del teclado.

3.3 Estructuras Repetitivas

El ciclo while running: constituye el bucle principal del juego (game loop). Este ciclo se ejecuta indefinidamente a 60 fotogramas por segundo, actualizando el estado del juego en cada iteración hasta que se cumple la condición de salida.

3.4 Funciones y Organización del Código

El código está organizado de forma clara y modular: cada bloque de código cumple una responsabilidad única (manejo de eventos, movimiento, colisiones, puntuación y renderizado), siguiendo el principio de separación de responsabilidades.

3.5 Herramientas de Desarrollo

Se utilizó Python 3.12 como lenguaje de programación, la biblioteca Pygame para la gestión gráfica y de eventos, y GitHub como sistema de control de versiones para registrar el historial de cambios del proyecto.

4. Cronograma de Actividades

El desarrollo del proyecto se distribuyó en ocho semanas de la siguiente manera:

Semana	Actividad	Descripción
Semana 1	Planificación del proyecto	Definición del alcance, identificación del problema, cronograma de actividades
Semana 2	Diseño funcional	Elaboración del diagrama de flujo lógico del sistema
Semana 3	Diseño de arquitectura	Desarrollo del diagrama de arquitectura de software
Semana 4	Desarrollo - Módulo principal	Implementación del bucle principal del juego y configuración de pygame
Semana 5	Desarrollo - Física y colisiones	Lógica de movimiento de la pelota, detección de colisiones con paletas y bordes
Semana 6	Desarrollo - Puntuación y UI	Sistema de puntuación, marcador en pantalla y condición de victoria
Semana 7	Pruebas y control de versiones	Pruebas funcionales del sistema, corrección de errores, commits en GitHub
Semana 8	Entrega final	Documentación completa, README, presentación y video demostrativo

5. Explicación del Sistema Desarrollado

5.1 Arquitectura del Sistema

El sistema sigue una arquitectura modular compuesta por seis componentes principales que interactúan entre sí a través del Motor del Juego:

- Interfaz de Usuario: muestra el juego en pantalla, recibe entradas del teclado y presenta el marcador.
- Motor del Juego: controla el ciclo principal, coordina todos los módulos y gestiona el estado del juego.
- Módulo de Física y Colisiones: calcula el movimiento de la pelota y detecta colisiones con paletas y bordes.
- Módulo de Puntuación: registra los puntos, actualiza el marcador y determina al ganador.
- Pelota y Paletas: objetos del juego que se mueven e interactúan en la pantalla.
- Marcador de Puntos: elemento visual que muestra la puntuación actual de cada jugador.

5.2 Diagrama de Flujo Lógico

El diagrama de flujo describe el ciclo de vida del juego desde el inicio hasta la pantalla de victoria. El flujo comienza con la inicialización de variables y la creación de los elementos del juego. Luego entra en el bucle principal donde: (1) lee la entrada del usuario, (2) mueve la pelota, (3) verifica colisiones, (4) comprueba si se anotó un punto, y (5) actualiza la pantalla. El ciclo termina cuando un jugador alcanza la puntuación de victoria.

5.4 Fragmento Principal del Código

```
# Requiere pygame instalado: pip install pygame y Python 3.12 Prueba
import pygame

pygame.init()

screen = pygame.display.set_mode((800, 600)) #Tamaño de la ventana

clock = pygame.time.Clock() #Frames por segundo del juego

font = pygame.font.Font(None, 75) #Tamaño de la fuente para el marcador
```



```
ball = pygame.Rect(400, 300, 20, 20) #Tamaño de la pelota
player1 = pygame.Rect(50, 250, 10, 100) #Tamaño de la paleta del jugador 1
player2 = pygame.Rect(740, 250, 10, 100) #Tamaño de la paleta del jugador 2
ball_speed = [5, 5] #Velocidad de la pelota
score1, score2 = 0, 0 #Marcador de los jugadores

running = True
while running:
    # Eventos
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False

    # Movimiento paletas
    keys = pygame.key.get_pressed()
    if keys[pygame.K_w]: player1.y -= 5 #Movimiento hacia arriba de la paleta del jugador 1
    if keys[pygame.K_s]: player1.y += 5 #Movimiento hacia abajo de la paleta del jugador 1
    if keys[pygame.K_UP]: player2.y -= 5 #Movimiento hacia arriba de la paleta del jugador 2
    if keys[pygame.K_DOWN]: player2.y += 5 #Movimiento hacia abajo de la paleta del jugador 2

    # Movimiento pelota
    ball.x += ball_speed[0] #Movimiento horizontal de la pelota
    ball.y += ball_speed[1] #Movimiento vertical de la pelota

    # Colisiones
    if ball.top <= 0 or ball.bottom >= 600: #Rebote de la pelota en los bordes superior e inferior
        ball_speed[1] = -ball_speed[1] #Rebote vertical
    if ball.colliderect(player1) or ball.colliderect(player2): #Rebote de la pelota en las paletas
        ball_speed[0] = -ball_speed[0] #Rebote horizontal
```



```
# Puntos

if ball.left <= 0: #Si la pelota sale por la izquierda, el jugador 2 anota un punto
    score2 += 1 #Si la pelota sale por la izquierda, el jugador 2 anota un punto
    ball.center = (400, 300) #Reinicia la posición de la pelota al centro de la pantalla

if ball.right >= 800: #Si la pelota sale por la derecha, el jugador 1 anota un punto
    score1 += 1 #Si la pelota sale por la derecha, el jugador 1 anota un punto
    ball.center = (400, 300) #Reinicia la posición de la pelota al centro de la pantalla

#victoria

if score1 == 5 or score2 == 5:
    winner_text = "Jugador 1 gana!" if score1 == 5 else "Jugador 2 gana!"
    text = font.render(winner_text, True, (255,255,255))
    screen.blit(text, (200, 250))
    pygame.display.flip()

    pygame.time.wait(3000) # Espera 3 segundos mostrando el mensaje

    running = False

# Dibujar

screen.fill((0, 0, 0))

pygame.draw.rect(screen, (255, 255, 255), player1) #Dibuja la paleta del jugador 1
pygame.draw.rect(screen, (255, 255, 255), player2) #Dibuja la paleta del jugador 2
pygame.draw.ellipse(screen, (255, 255, 255), ball) #Dibuja la pelota

# Marcador

text1 = font.render(str(score1), True, (255,255,255)) #Dibuja el marcador del jugador 1
text2 = font.render(str(score2), True, (255,255,255)) #Dibuja el marcador del jugador 2
screen.blit(text1, (250, 10)) #Dibuja el marcador del jugador 1
screen.blit(text2, (500, 10)) #Dibuja el marcador del jugador 2

pygame.display.flip() #Actualiza la pantalla

clock.tick(60) #Frames por segundo del juego
```

```
pygame.quit()
```

Conclusión

- El proyecto permitió aplicar de forma integral los contenidos de la asignatura: variables, estructuras condicionales, estructuras repetitivas y organización del código.
- El diagrama de flujo y el diagrama de arquitectura facilitaron la comprensión del sistema antes de iniciar la implementación, reduciendo errores durante el desarrollo.
- Python y Pygame resultaron herramientas adecuadas para el nivel de la asignatura, permitiendo obtener resultados visuales de forma rápida y con código comprensible.
- GitHub como herramienta de control de versiones demostró ser fundamental para mantener un historial ordenado del proyecto y facilitar la colaboración futura.
- El ejercicio de programar un videojuego clásico refuerza la comprensión de que la tecnología es una herramienta construida por personas para resolver problemas y generar valor en la sociedad.

Bibliografía

Pygame Community. (2024). Pygame Documentation. <https://www.pygame.org/docs/>

Python Software Foundation. (2024). Python 3.12 Documentation. <https://docs.python.org/3/>

Sommerville, I. (2016). Software Engineering (10th ed.). Pearson Education.

GitHub, Inc. (2024). GitHub Docs. <https://docs.github.com/>



6. Reflexión sobre el Impacto de la Tecnología

6.1 Impacto del Software en la Sociedad

Created by
Arizona State University®

El desarrollo de software, incluso a nivel educativo, genera un impacto significativo en la sociedad. Los videojuegos, como Atari Pong, fueron pioneros en democratizar el entretenimiento digital y abrieron el camino para la industria tecnológica tal como la conocemos hoy. La programación de juegos introduce conceptos que luego se aplican en simulaciones médicas, sistemas de control industrial, inteligencia artificial y muchas otras áreas críticas.

6.2 La Programación como Herramienta Transformadora

Aprender a programar no es únicamente adquirir una habilidad técnica: es desarrollar una forma de pensamiento lógico, sistemático y orientado a la solución de problemas. Este proyecto demuestra cómo conceptos básicos como variables, condicionales y bucles pueden combinarse para crear una experiencia interactiva completa.

La capacidad de traducir un problema del mundo real en instrucciones que una computadora puede ejecutar es la habilidad fundamental de la era digital. En un mundo cada vez más automatizado, esta competencia tiene un valor transformador tanto a nivel personal como social.

6.3 Responsabilidad del Desarrollador

El desarrollo de software conlleva también una responsabilidad ética. Los programadores toman decisiones que afectan a los usuarios: la accesibilidad de las interfaces, la seguridad de los datos y el impacto ambiental de los sistemas. Incluso en un proyecto educativo como este, se practica la responsabilidad de escribir código claro, documentado y funcional.

7. Conclusiones

- El proyecto permitió aplicar de forma integral los contenidos de la asignatura: variables, estructuras condicionales, estructuras repetitivas y organización del código.
- El diagrama de flujo y el diagrama de arquitectura facilitaron la comprensión del sistema antes de iniciar la implementación, reduciendo errores durante el desarrollo.
- Python y Pygame resultaron herramientas adecuadas para el nivel de la asignatura, permitiendo obtener resultados visuales de forma rápida y con código comprensible.
- GitHub como herramienta de control de versiones demostró ser fundamental para mantener un historial ordenado del proyecto y facilitar la colaboración futura.



- El ejercicio de programar un videojuego clásico refuerza la comprensión de que la tecnología es una herramienta construida por personas para resolver problemas y generar valor en la sociedad.